



400-650-7088

***Tong*RDS** V2.2

场景配置参考手册

版本：2218A01

2025 年 9 月

声明

版权

Copyright ©2025 北京东方通科技股份有限公司或其关联公司（以下简称“东方通”）。保留所有权利。

版权声明

- 本文档的所有权和知识产权归属于东方通所有。
- 除非另有明确约定，东方通不授予任何明示或暗示的许可或权利，使用者不得以任何方式将本文档中的任何内容用于商业目的。
- 东方通对本文档享有版权。本文档的所有内容，包括但不限于文字、图像、图表、图标、示意图、屏幕截图等，均受版权法和国际版权条约的保护。
- 未经东方通明确授权，任何人不得对本文档以任何形式复制、修改、传播、分发、展示或进行衍生创作。

商标声明

- **东方通**、**TongTech**标识以及其他相关东方通图形、徽标、服务名称和商标（以下统称为“东方通商标”）是东方通或其关联公司的注册商标或商标。
- 未经东方通明确授权，任何人不得以任何方式使用、复制或展示东方通商标。此外，未经东方通事先书面同意，不得将东方通商标与其他商标、标识或徽标进行混淆、链接或结合使用。
- 除非另有明确约定，本商标声明不授予任何明示或暗示的许可或权利，对东方通商标的使用须获得东方通的明确授权。
- 本文档提及的所有其他商标、标识、徽标或产品名称均为其各自所有者的财产，并可能受其各自的商标法保护。

免责声明

请在使用东方通产品之前仔细阅读本免责声明，并根据自身情况判断是否继续使用。如有任何问题，请联系我们的客户支持团队。

- 本产品的使用是基于用户自己的判断和风险评估。本文档仅作为使用指导，不对使用本产品所产生的结果做任何明示或暗示的担保。
- 东方通不对用户未按照本文档中的指导正确使用东方通产品而导致的任何损失或损害承担责任；
- 本文档可能会包含第三方提供的内容、链接或资源，这些内容由第三方自行负责。东方通对于这些内容的准确性、完整性、合法性或可靠性不承担责任。用户在使用这些内容时应自行判断和承担风险。
- 由于产品版本升级或其他原因，本文档内容会不定期更新。东方通保留在不事先通知用户的情况下随时对文档进行修改、更新或中止的权利。

如需获取有关东方通产品的许可或使用权，请联系东方通或授权代理商。任何违反本声明的行为将受到适用法律的追究。

版本变更说明

本手册的更新是累积的。因此，最新的手册版本包含对以前版本所做的所有更改。

本手册版本（2218A01）适用于 TongRDS V2.2.1.8。

文档版本	适用产品版本	更新内容	更新日期
2218A01	V2.2.1.8	更新如下内容： 更新版权声明。	2025/9/29
2217P1A01	V2.2.1.7_P1	无	2025/6/13
2217A01	V2.2.1.7	无	2025/4/18
2216P2A01	V2.2.1.6_P2	无	2024/12/6
2216P1A01	V2.2.1.6_P1	无	2024/9/29
2216A02	V2.2.1.6	新增如下内容： ● 4.1.2 部署哨兵节点：新增章节； ● 4.1.5 启动服务：新增启动哨兵服务的步骤。 更新如下内容： 4.1.3 部署服务节点：更新配置步骤。	2024/9/6
2216A01	V2.2.1.6	无	2024/8/8
2215A01	V2.2.1.5	新增如下内容： 第3章 代理集群模式：新增章节。	2024/6/28
2214A01	V2.2.1.4	TongRDS V2.2第一次正式发布。	2023/12/29

前言

本文档是 TongRDS V2.2 产品的用户使用手册之一，提供常见部署模式的配置示例及验证方法。

阅读前注意事项

通过阅读本文档，您确认并同意自行承担因未具备必要专业背景 and 知识而导致的任何风险或后果。在使用本文档中提供的信息和指南时，请始终谨慎，并在必要时寻求专业人士建议和指导。

● 适用对象

本文档主要适用于使用本产品的系统管理员阅读，部分内容同样适用于基于本产品进行应用开发或应用部署的人员阅读。

● 专业背景

本文内容可能涉及到操作系统、服务器硬件、网络等相关领域的知识。请确保您具备相关背景 and 知识，以便更好地理解 and 应用本手册的内容。

● 技能要求

为了能够充分理解 and 应用本文档的内容，建议您具备如下技能：

1. 掌握 Linux 系统基本操作；
2. 熟悉 Java 开发语言。

● 术语和概念

本文档可能使用一些专业术语和概念。请确保您熟悉这些术语和概念，或者有能力查阅相关资料以便进一步理解。

● 实践经验

为了最大程度地受益于本文档，建议您具备一定的实践经验。这将帮助您更好地应用文档中的操作指南和建议。

请注意，本文档并不适用于没有相关专业背景 and 知识的用户。如果您对本文档的内容或所需背景有任何疑问，请在使用之前咨询相关专业人士或寻求额外的支持。

用户使用手册集

TongRDS V2.2 为您提供的用户使用手册集包含的文档有：

编号	手册集文档	说明
1	000_TongRDS_V2.2 用户手册导读	提供手册集的目录及手册版本说明。
2	001_TongRDS_V2.2 快速使用手册	提供产品介绍，安装部署产品的基本步骤及验证方式。
3	002_TongRDS_V2.2 配置参数手册	提供配置文件及参数的详细说明。

编号	手册集文档	说明
4	003_TongRDS_V2.2 管理控制台安装手册	详细介绍管理控制台的安装卸载、启动停止及登录。
5	004_TongRDS_V2.2 管理控制台使用手册	提供管理控制台基础信息、RDS 管理、系统监控、系统审计等功能的使用说明。
6	005_TongRDS_V2.2 客户端使用手册	介绍 TongRDS 提供的客户端程序使用说明。
7	006_TongRDS_V2.2 监控管理手册	提供安装部署监控管理组件的基本步骤。
8	007_TongRDS_V2.2 场景配置参考手册	提供常见部署模式的配置示例及验证方法。
9	008_TongRDS_V2.2 命令&接口使用手册	提供命令及接口的详细说明及使用方法。
10	009_TongRDS_V2.2 开发手册	详细介绍 TongRDS 提供的开发功能。
11	010_TongRDS_V2.2 运维手册	提供 TongRDS 常见问题的解决方案。

技术支持

东方通产品将为您提供全方位的技术支持，您可以通过以下方式获得技术支持：

网址：www.tongtech.com

电话：400-650-7088

邮箱：support@tongtech.com

您在取得技术支持时，请提供如下信息：

1. 您的姓名
2. 您的公司信息
3. 您的联系方式
4. 硬件及软件信息
5. 操作系统及其版本
6. 产品版本号
7. 日志等错误的详细信息

目录

版本变更说明.....	3
前言.....	4
目录.....	6
第 1 章 概述	7
第 2 章 集群模式.....	8
2.1 部署前准备	8
2.2 部署服务节点	9
2.3 部署中心节点	10
2.4 启动集群	11
2.5 部署后验证	13
第 3 章 代理集群模式.....	16
3.1 部署前准备	16
3.2 部署代理节点	17
3.3 部署服务节点	19
3.4 部署中心节点	20
3.5 启动集群	22
3.6 部署后验证	23
第 4 章 哨兵模式.....	26
4.1 独立哨兵服务配置	26
4.1.1 部署前准备	26
4.1.2 部署哨兵节点	26
4.1.3 部署服务节点	30
4.1.4 部署中心节点	31
4.1.5 启动服务	33
4.1.6 客户端 jedis（以 Version3.5.0 为例）连接.....	34
4.2 Center 哨兵配置.....	35
4.2.1 部署前准备	35
4.2.2 部署服务节点	35
4.2.3 部署中心节点	37
4.2.4 启动服务	38
4.2.5 部署后验证	39
第 5 章 故障排除.....	42
5.1 主备（主从）节点数据不同步	42
附录 A 术语说明	43

第1章 概述

TongRDS 是分布式内存数据缓存中间件，用于高性能内存数据共享与应用支持。TongRDS 为各类应用提供高效、稳定、安全的内存数据处理能力；同时它支持共享内存的搭建弹性伸缩管理；使业务应用无需考虑各种内存的复杂管理。

服务节点：用于提供基础的 TongRDS 数据管理服务、哨兵监控服务，通过 TongRDS 服务节点的组合（即集群模式：单节点、哨兵集群、集群模式）可以提供完整的数据缓存服务。

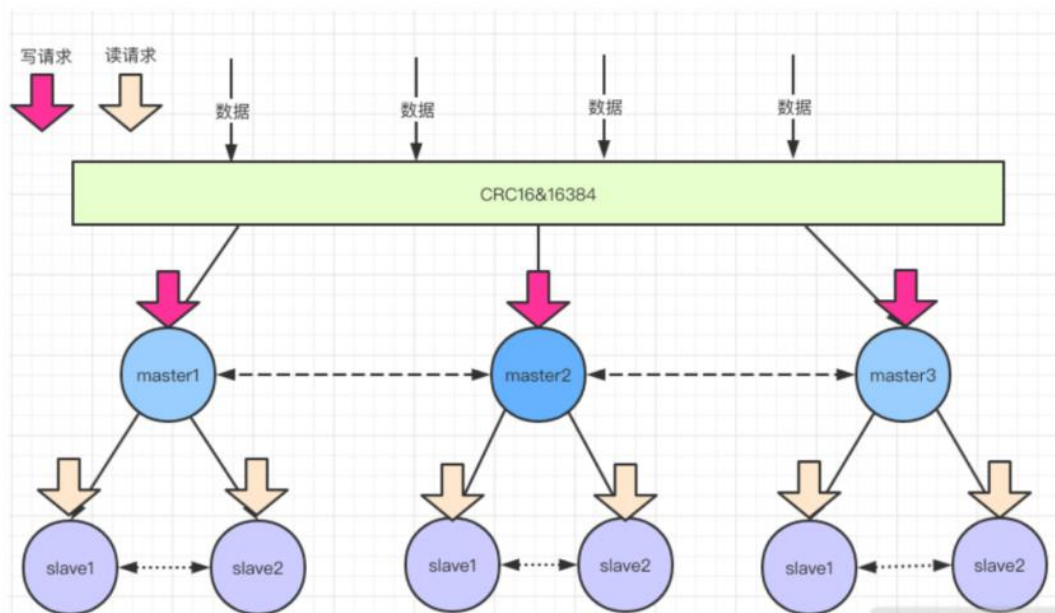
中心节点：是缓存中间件企业版提供的集中管理模块，负责各服务节点以及接入的客户端的集中监控管理，配置同步，License 控制等集中管理操作。

本手册介绍常见的部署模式配置示例，包括如下部署模式：

- 集群模式
- 代理集群模式
- 哨兵模式：
 - 独立哨兵服务配置
 - Center 哨兵配置

第2章 集群模式

RDS Cluster 集群能起到很好的负载均衡的目的。集群节点最小可配置 2 个节点（1 主 1 从），通常配置 6 个节点（3 主 3 从），其中主节点提供读写操作，从节点作为备用节点同时提供读操作和故障转移使用，本场景采用 3 主 3 从配置部署。集群中采用虚拟槽分区，所有的键根据哈希函数映射到 0~16383 个整数槽内，每个节点负责维护一部分槽以及槽所映射的键值数据，如下图所示：



2.1 部署前准备

部署 TongRDS 的集群模式，至少需要部署 1 个中心节点，3 个作为集群中主节点的服务节点和 3 个作为集群中从节点的服务节点。

在安装 TongRDS 前，需要在服务器上创建操作系统账号 rds，HOME 目录为/home/rds。

以下步骤中 TongRDS 均安装在此账号下。

TongRDS 集群部署示例中使用的 IP 如下表所示：

部署类型		目录	IP	Port
中心节点		/home/rds/pcenter	192.168.0.86	6300
			192.168.0.87（可选）	
服务节点	主	/home/rds/pmemdb	192.168.0.10	TongRDS 协议端口：6200 Redis 仿真端口：6379
			192.168.0.20	
			192.168.0.30	
	从		192.168.0.11	
			192.168.0.21	
			192.168.0.31	

2.2 部署服务节点

1. 获取服务节点安装包，将安装包分别上传到 6 台服务器的 **/home/rds** 目录下。
2. 进入 **/home/rds** 目录，执行解压命令：

```
tar zxvf TongRDS-2.2.x.x.Node.tar.gz
```

3. 分别进入 6 台服务器的配置文件目录 **pmemdb/etc**

```
cd pmemdb/etc
```

4. 修改配置文件 **dynamic.xml**，6 台服务器上的配置都需要修改。

```
vi dynamic.xml
```

```
<Server>
  <Center>
    <Password>454d51192b1704c60e19734ce6b38203</Password>
    <EndPoint>
      <Host>192.168.0.86</Host>
      <Port>6300</Port>
    </EndPoint>
  </Center>
</Server>
```

参数说明：

- **Server.Center.Password**：连接 Center 时的认证密码，示例中使用默认值即可。
- **Sever.Center.EndPoint**：中心节点服务器 IP 地址，示例中修改 IP 地址为 192.168.0.86，端口为 6300。如果存在多台 Center 节点，EndPoint 可配置多条，例如：

```
<Server>
  <Center>
    <Password>454d51192b1704c60e19734ce6b38203</Password>
    <EndPoint>
      <Host>192.168.0.86</Host>
      <Port>6300</Port>
    </EndPoint>
```

```
<EndPoint>

    <Host>192.168.0.87</Host>

    <Port>6300</Port>

</EndPoint>

</Center>

</Server>
```

2.3 部署中心节点

1. 获取中心节点安装包，将安装包上传到服务器的/home/rds 目录下。
2. 进入/home/rds 目录，执行解压命令：

```
tar zxvf TongRDS-2.2.x.x.MC.tar.gz
```

3. 将 License 文件 **center.lic** 复制到中心节点的 **pcenter** 目录下。
4. 进入配置文件目录 pcenter/etc

```
cd pcenter/etc
```

5. 在配置文件 **config.properties** 中检查如下配置项的值：

```
vi config.properties
```

```
...

# Service 节点连接 Center 节点使用的密码

server.password=454d51192b1704c60e19734ce6b38203

...

# 服务器监听端口，用于接收 MemDB 的上报数据

server.service.port=6300

...
```

- **server.password:** 节点接入时的认证密码，采用 SM4 加密，与服务节点 dynamic.xml 中的 Server.Center.Password 保持一致，示例中使用默认值即可。
- **server.service.port:** 中心节点的主服务端口，与服务节点 dynamic.xml 中的 Sever.Center.Endpoint.Port 保持一致，默认为 6300，示例中使用默认值即可。

6. 修改配置文件 **cluster.properties** 为如下配置：

```
vi cluster.properties
```

```
WebSession.type=cluster

WebSession.shards=3

WebSession.shard0.nodes=192.168.0.10:6200, 192.168.0.11:6200

WebSession.shard0.slots=0-5000

WebSession.shard1.nodes=192.168.0.20:6200, 192.168.0.21:6200

WebSession.shard1.slots=5001-10000

WebSession.shard2.nodes=192.168.0.30:6200, 192.168.0.31:6200

WebSession.shard2.slots=10001-16383
```

- **WebSession.type=cluster:** 服务 “WebSession”（对应服务节点中的 Server.Service=WebSession 配置）的状态为集群（cluster）模式。
- **WebSession.shards:** 定义了服务 “WebSession” 包含 3 个服务节点分组，WebSession.shard0.nodes 定义第一个 cluster 分组的服务节点地址，排在前面的是主节点，后面的是从节点（此处的配置需要和实际的服务节点的运行情况对应）；WebSession.shard0.slots 定义第一个分组处理的数据切片的范围。

7. （可选）如果有多台中心节点集群，可在配置文件 **sync.properties** 中配置多台中心节点。例如配置 2 台 Center 集群：

```
sync.servers=2

sync.server1.host=192.168.0.86

sync.server1.port=6300

sync.server2.host=192.168.0.87

sync.server2.port=6300
```

当前示例中可不配置此文件。

2.4 启动集群

1. 在中心节点所在服务器的 **pcenter/bin** 目录下运行以下命令启动中心节点：

./StartCenter.sh

```
[rds@master bin]# ./StartCenter.sh

CenterService start at 6300

SentinelService start at 26379

Center start.
```

```
RestServer start at 8086
```

2. 分别在 6 台服务节点所在服务器的 **pmemdb/bin** 目录下运行以下命令启动服务节点：

./StartServer.sh

```
[rds@master bin]# ./StartServer.sh

load config-file '/home/rds/pmemdb/etc/cfg.xml' ok.

Server belonging to 'WebSession' is starting...

Memory cache create ok.

Start listening to port 6200

Start listening to port 6379

Server started.
```

服务节点启动成功会连接 Center 节点并获得 Cluster 配置，各服务节点的 **dynamic.xml** 文件将被修改为类似如下配置：

```
<Server>
  <Center>
    <Password>454d51192b1704c60e19734ce6b38203</Password>
    <EndPoint>
      <Host>192.168.0.86</Host>
      <Port>6300</Port>
    </EndPoint>
  </Center>
  <Synchronize>
    <EndPoint>
      <Host>192.168.0.10</Host>
      <Port>6200</Port>
    </EndPoint>
    <EndPoint>
```

```

        <Host>192.168.0.11</Host>

        <Port>6200</Port>

    </EndPoint>

</Synchronize>

<Clusters>

    <Cluster>

        <EndPoints>192.168.0.10:6200, 192.168.0.11:6200</EndPoints>

        <Slots>0-5000</Slots>

    </Cluster>

    <Cluster>

        <EndPoints>192.168.0.20:6200, 192.168.0.21:6200</EndPoints>

        <Slots>5001-10000</Slots>

    </Cluster>

    <Cluster>

        <EndPoints>192.168.0.30:6200, 192.168.0.31:6200</EndPoints>

        <Slots>10001-16383</Slots>

    </Cluster>

</Clusters>

</Server>

```

2.5 部署后验证

采用 jedis(版本 3.7.0)连接 RDS 集群,并进行简单的数据读写操作,验证集群的可用性以及仿真 Redis。

新建一个 java 对象, 加入如下 main 方法并执行:

```

public static void main(String[] args){

    HashSet<HostAndPort> nodes = new HashSet<>();

    nodes.add(new HostAndPort("192.168.0.10", 6379));

    nodes.add(new HostAndPort("192.168.0.11", 6379));

    JedisCluster jedisCluster = new JedisCluster(nodes, 1000, 1000, 3, "123", new GenericObjectPoolConfig());
}

```

```
System.out.println(jedisCluster.set("aaa", "1001"));
```

```
System.out.println(jedisCluster.get("aaa"));
```

```
System.out.println(jedisCluster.set("aba", "1002"));
```

```
System.out.println(jedisCluster.get("aba"));
```

```
System.out.println(jedisCluster.set("abc", "1003"));
```

```
System.out.println(jedisCluster.get("abc"));
```

```
System.out.println(jedisCluster.set("bbb", "1011"));
```

```
System.out.println(jedisCluster.get("bbb"));
```

```
System.out.println(jedisCluster.set("bab", "1012"));
```

```
System.out.println(jedisCluster.get("bab"));
```

```
System.out.println(jedisCluster.set("bac", "1013"));
```

```
System.out.println(jedisCluster.get("bac"));
```

```
System.out.println(jedisCluster.set("bbc", "1014"));
```

```
System.out.println(jedisCluster.get("bbc"));
```

```
System.out.println(jedisCluster.set("ccc", "1021"));
```

```
System.out.println(jedisCluster.get("ccc"));
```

```
System.out.println(jedisCluster.set("cac", "1022"));
```

```
System.out.println(jedisCluster.get("cac"));
```

```
System.out.println(jedisCluster.set("cbc", "1023"));
```

```
System.out.println(jedisCluster.get("cbc"));

jedisCluster.subscribe(new MyPubsub(),"channel");

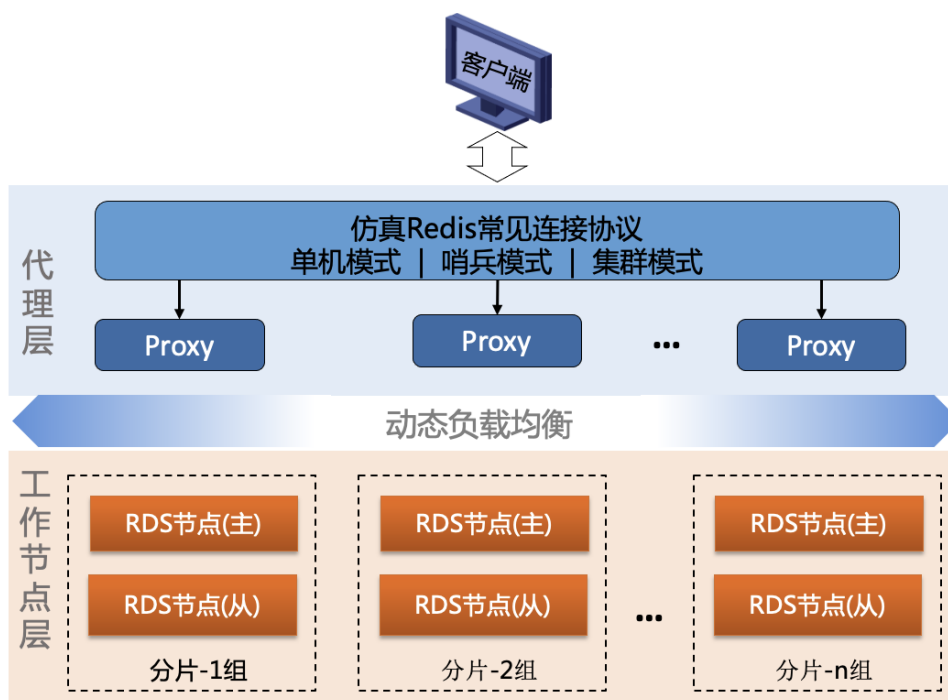
jedisCluster.publish("channel","aaa");

jedisCluster.close();

}
```

第3章 代理集群模式

代理集群模式由内部 proxy 集群提供数据的动态负载均衡和接入协议的隔离。客户端无需考虑工作节点层的集群内部结构和动态变化，代理层会自动跟踪工作节点的变化做出优化调整，保证最优的数据负载分配。代理层隔离了内部复杂的集群连接协议，Proxy 机制提供统一单服务接入协议，也提供不变的连接地址和接口，保证了客户端对接的一致性。为了兼容不同连接模式的客户端，代理层支持单机模式、哨兵模式、集群模式这三种常见的连接协议。对多种连接模式的兼容，也保证了迁移到 RDS 时无需对客户端进行任何修改。如下图所示：



代理层：由内部 proxy 集群提供数据的动态负载均衡，客户端无需考虑工作节点层的集群内部结构和动态变化，代理层会自动跟踪工作节点的变化。

工作节点层：提供数据缓存的核心数据服务，它组建了一个类似 Redis 集群模式的服务；通过中心服务的调度，使用其具有服务自动编排和资源节点动态伸缩的能力。

3.1 部署前准备

部署 TongRDS 的代理集群模式，至少需要部署 1 个中心节点，1 个代理节点和 2 个分别作为主节点和从节点的服务节点。

在安装 TongRDS 前，需要在服务器上创建操作系统账号 rds，HOME 目录为/home/rds。

以下步骤中 TongRDS 均安装在此账号下。

TongRDS 代理集群模式部署示例中使用的 IP 如下表所示：

部署类型	目录	IP	Port
中心节点	/home/rds/pcenter	192.168.0.86	6300

部署类型	目录	IP	Port
		192.168.0.87（可选）	
代理节点	/home/rds/proxy	192.168.0.10	TongRDS 协议端口：6200 Redis 仿真端口：6379
服务节点	/home/rds/pmemdb	192.168.0.20	TongRDS 协议端口：6200 Redis 仿真端口：6379
		192.168.0.30	

3.2 部署代理节点

1. 获取代理节点安装包，将安装包上传到服务器的/home/rds 目录下。
2. 进入/home/rds 目录，执行解压命令：

```
tar zxvf TongRDS-2.2.x.x.Proxy.tar.gz
```

3. 进入服务器的配置文件目录 proxy/etc

```
cd proxy/etc
```

4. 修改配置文件 dynamic.xml

```
vi dynamic.xml
```

```
<Server>
  <Center>
    <Password>454d51192b1704c60e19734ce6b38203</Password>
    <EndPoint>
      <Host>192.168.0.86</Host>
      <Port>6300</Port>
    </EndPoint>
  </Center>
</Server>
```

- **Server.Center.Password:** 连接 Center 时的认证密码，示例中使用默认值即可。
- **Sever.Center.EndPoint:** 中心节点服务器 IP 地址，示例中修改 IP 地址为 192.168.0.86，端口为 6300。如果存在多台 Center 节点，EndPoint 可配置多条，例如：

```
<Server>
  <Center>
    <Password>454d51192b1704c60e19734ce6b38203</Password>
```

```

        <EndPoint>

            <Host>192.168.0.86</Host>

            <Port>6300</Port>

        </EndPoint>

        <EndPoint>

            <Host>192.168.0.87</Host>

            <Port>6300</Port>

        </EndPoint>

    </Center>

</Server>

```

5. 修改配置文件 proxy.xml

```

<Server>

    <Common>

        <Instance>proxy</Instance>

        <PretendCluster>true</PretendCluster>

    </Common>

    <Listen>

        <Port>6200</Port>

        <Secure>2</Secure>

        <Password>8cd42d74926b50082f6655a4b07c6458</Password>

        <RedisPort>6379</RedisPort>

    </Listen>

</Server>

```

参数说明：

- **Server.Common.Instance:** 实例名，即当前代理节点的名称，需要保证全系统唯一。
- **Server.Common.PretendCluster:** 将 proxy 模拟成单分片的集群。
- **Server.Listen.Port:** 代理节点内部数据交换端口，默认为 6200。
- **Server.Listen.Secure:** 连接安全等级，2 为需要密码认证的非加密连接，3 为需要密码认证的 SSL 加密连接。

- **Server.Listen.Password:** SM4 加密的密码。
- **Server.Listen.RedisPort:** Redis 仿真端口，默认为 6379。

3.3 部署服务节点

1. 获取服务节点安装包，将安装包分别上传到 2 台服务器的 **/home/rds** 目录下。
2. 进入 **/home/rds** 目录，执行解压命令：

```
tar zxvf TongRDS-2.2.x.x.Node.tar.gz
```

3. 分别进入 2 台服务器的配置文件目录 **pmemdb/etc**

```
cd pmemdb/etc
```

4. 修改配置文件 **cfg.xml**，在 **Common** 节中增加 **Instance** 配置，2 台服务器上的配置都需要修改且 **Instance** 值不能一样，例如第一个节点配置为 “rds-0”，第二个节点配置为 “rds-1”

```
<Server>

  <Common>

    <RuntimeModel>debug</RuntimeModel>

    <DataDump>${Server.Common.DataDump:10}</DataDump>

    <DataDumpAppending>${Server.Common.DataDumpAppending:false}</DataDumpAppending>

    <StartWaiting>${Server.Common.StartWaiting:5}</StartWaiting>

    <Service>${Server.Common.Service:WebSession}</Service>

    <MaxKeyLength>${Server.Common.MaxKeyLength:1m}</MaxKeyLength>

    <MaxValueLength>${Server.Common.MaxValueLength:10m}</MaxValueLength>

    <Instance>rds-0</Instance>

  </Common>

  .....
```

5. 修改配置文件 **dynamic.xml**，2 台服务器上的配置都需要修改。

```
vi dynamic.xml
```

```
<Server>

  <Center>

    <Password>454d51192b1704c60e19734ce6b38203</Password>
```

```
<EndPoint>

  <Host>192.168.0.86</Host>

  <Port>6300</Port>

</EndPoint>

</Center>

</Server>
```

参数说明：

- **Server.Center.Password:** 连接 Center 时的认证密码，示例中使用默认值即可。
- **Sever.Center.EndPoint:** 中心节点服务器 IP 地址，示例中修改 IP 地址为 192.168.0.86，端口为 6300。如果存在多台 Center 节点，EndPoint 可配置多条，例如：

```
<Server>

  <Center>

    <Password>454d51192b1704c60e19734ce6b38203</Password>

    <EndPoint>

      <Host>192.168.0.86</Host>

      <Port>6300</Port>

    </EndPoint>

    <EndPoint>

      <Host>192.168.0.87</Host>

      <Port>6300</Port>

    </EndPoint>

  </Center>

</Server>
```

3.4 部署中心节点

1. 获取中心节点安装包，将安装包上传到服务器的 **/home/rds** 目录下。
2. 进入 **/home/rds** 目录，执行解压命令：

```
tar zxvf TongRDS-2.2.x.x.MC.tar.gz
```

3. 将 License 文件 **center.lic** 复制到中心节点的 **pcenter** 目录下。

4. 进入配置文件目录 `pcenter/etc`

cd pcenter/etc

5. 在配置文件 **config.properties** 中检查如下配置项的值：

vi config.properties

```
...
# Service 节点连接 Center 节点使用的密码
server.password=454d51192b1704c60e19734ce6b38203
...
# 服务器监听端口，用于接收 MemDB 的上报数据
server.service.port=6300
...
```

- **server.password:** 节点接入时的认证密码，采用 SM4 加密，与服务节点 `dynamic.xml` 中的 `Server.Center.Password` 保持一致，示例中使用默认值即可。
- **server.service.port:** 中心节点的主服务端口，与服务节点 `dynamic.xml` 中的 `Sever.Center.Endpoint.Port` 保持一致，默认为 6300，示例中使用默认值即可。

6. 修改配置文件 **cluster.properties** 为如下配置：

vi cluster.properties

```
WebSession.type=adjustable
WebSession.shards=2
WebSession.shard0.nodes=rds-0
WebSession.shard0.slots=0-8193
WebSession.shard1.nodes=rds-1
WebSession.shard1.slots=8194-16383
```

- **WebSession.type=adjustable:** 服务“WebSession”（对应服务节点中的 `Server.Service=WebSession` 配置）的状态为代理集群（adjustable）模式。
- **WebSession.shards:** 集群分片数量。
- **WebSession.shard?.nodes:** 各分片中服务节点的实例名（对应配置的 `Instance` 项）。
- **WebSession.shard?.slots:** 各分片对应的数据槽位号。

7. （可选）如果有多台中心节点集群，可在配置文件 **sync.properties** 中配置多台中心节点。例如配置 2 台 Center 集群：

```
sync.servers=2

sync.server1.host=192.168.0.86

sync.server1.port=6300

sync.server2.host=192.168.0.87

sync.server2.port=6300
```

当前示例中可不配置此文件。

3.5 启动集群

1. 在中心节点所在服务器的 **pcenter/bin** 目录下运行以下命令启动中心节点：

./StartCenter.sh

```
[rds@master bin]# ./StartCenter.sh

CenterService start at 6300

SentinelService start at 26379

Center start.

RestServer start at 8086
```

2. 在代理节点所在服务器的 **proxy/bin** 目录下运行以下命令启动代理节点：

./StartProxy.sh

```
[rds@master bin]# ./StartProxy.sh

load config-file '/home/rds/proxy/etc/proxy.xml' ok.

Start listening to port 6200

Start listening to port 6379
```

3. 分别在 2 台服务节点所在服务器的 **pmemdb/bin** 目录下运行以下命令启动服务节点：

./StartServer.sh

```
[rds@master bin]# ./StartServer.sh

load config-file '/home/rds/pmemdb/etc/cfg.xml' ok.


Server belonging to 'WebSession' is starting...
```

```
Memory cache create ok.

Start listening to port 6200

Start listening to port 6379

Server started.
```

服务节点启动成功会连接 Center 节点并获得 Cluster 配置，各服务节点的 **dynamic.xml** 文件将被修改为类似如下配置：

```
<Server>

  <Center>

    <Password>454d51192b1704c60e19734ce6b38203</Password>

    <EndPoint>

      <Host>node1</Host>

      <Port>6300</Port>

    </EndPoint>

  </Center>

  <Synchronize>

    <EndPoint>

      <Host>192.168.0.20</Host>

      <Port>6200</Port>

    </EndPoint>

    <EndPoint>

      <Host>192.168.0.30</Host>

      <Port>6200</Port>

    </EndPoint>

  </Synchronize>

</Server>
```

3.6 部署后验证

通过 PretendCluster 配置指定 proxy 代理节点仿真为传统 Cluster 集群模式，可采用集群方式连接代理节点验证。

采用 jedis(版本 3.7.0)连接 RDS 集群,并进行简单的数据读写操作,验证集群的可用性以及仿真 Redis。

新建一个 java 对象, 加入如下 main 方法并执行:

```
public static void main(String[] args){

    HashSet<HostAndPort> nodes = new HashSet<>();

    nodes.add(new HostAndPort("192.168.0.10", 6379));

    JedisCluster jedisCluster = new JedisCluster(nodes, 1000, 1000, 3, "123", new GenericObjectPoolConfig());

    System.out.println(jedisCluster.set("aaa", "1001"));
    System.out.println(jedisCluster.get("aaa"));

    System.out.println(jedisCluster.set("aba", "1002"));
    System.out.println(jedisCluster.get("aba"));

    System.out.println(jedisCluster.set("abc", "1003"));
    System.out.println(jedisCluster.get("abc"));

    System.out.println(jedisCluster.set("bbb", "1011"));
    System.out.println(jedisCluster.get("bbb"));

    System.out.println(jedisCluster.set("bab", "1012"));
    System.out.println(jedisCluster.get("bab"));

    System.out.println(jedisCluster.set("bac", "1013"));
    System.out.println(jedisCluster.get("bac"));

    System.out.println(jedisCluster.set("bbc", "1014"));
    System.out.println(jedisCluster.get("bbc"));
```



```

        System.out.println(jedisCluster.set("ccc", "1021"));

        System.out.println(jedisCluster.get("ccc"));

        System.out.println(jedisCluster.set("cac", "1022"));

        System.out.println(jedisCluster.get("cac"));

        System.out.println(jedisCluster.set("cbc", "1023"));

        System.out.println(jedisCluster.get("cbc"));

        jedisCluster.subscribe(new MyPubsub(),"channel");

        jedisCluster.publish("channel","aaa");

        jedisCluster.close();
    }

```

第4章 哨兵模式

4.1 独立哨兵服务配置

RDS 提供独立的哨兵组件。独立的哨兵组件包含在服务节点包中随服务节点发行。相比中心节点附带的哨兵功能，该模块部署更灵活，客户端适应性更强（例如：中心节点的哨兵功能要求登录必须使用密码，但有的早期客户端不支持带密码的哨兵，独立的哨兵组件允许无密码接入）。

使用该组件可帮助用户更灵活的解决 redis 时需要哨兵的问题，独立模块哨兵有如下特点：

- 独立于 RDS-Node 节点运行；
- 运行时无需 license 授权；
- 根据配置主动检测服务节点状态；
- 通过配置，一个哨兵可同时监控多个 RDS 服务组；
- 可同时运行多个哨兵程序，根据各自的配置文件独立运行。

4.1.1 部署前准备

独立哨兵服务配置通常需要部署 1 个中心节点，2 个服务节点，1 个哨兵节点，哨兵服务（独立的哨兵服务与服务节点一同打包发行。开启独立哨兵需安装服务节点软件并启动其中的独立哨兵服务）。

在安装 TongRDS 前，需要在服务器上创建操作系统账号 rds，HOME 目录为/home/rds。

以下步骤中 TongRDS 均安装在此账号下。

TongRDS 独立哨兵服务配置示例中使用的 IP 如下表所示：

部署类型	目录	IP	Port
中心节点	/home/rds/pcenter	192.168.0.86	6300
		192.168.0.87（可选）	
服务节点	/home/rds/pmemdb	192.168.0.10	TongRDS 协议端口：6200 Redis 仿真端口：6379
		192.168.0.20	
哨兵节点	/home/rds/sentinel	192.168.0.10	哨兵监听端口：26379

4.1.2 部署哨兵节点

1. 获取服务节点安装包，将安装包上传到其中 1 台服务器的/home/rds 目录下。
2. 进入/home/rds 目录，执行解压命令：

```
tar zxvf TongRDS-2.2.x.x.Node.tar.gz
```

3. 将解压后的目录名称修改为 sentinel。

```
mv pmemdb/sentinel
```

4. 修改配置文件 **sentinel.xml**。

vi sentinel.xml

```
<Server>

  <Common>

    <SslCiphers>

      TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384, TLS_ECDH_ECDSA_W
      ITH_AES_256_GCM_SHA384, TLS_ECDHE_ECDSA_WITH_AES_256_CBC
      _SHA, TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA, TLS_RSA_WITH_A
      ES_256_CBC_SHA, TLS_KRB5_WITH_DES_CBC_MD5, TLS_KRB5_WITH
      _3DES_EDE_CBC_SHA, TLS_ECDHE_RSA_WITH_3DES_EDE_CBC_SHA,
      TLS_ECDH_ECDSA_WITH_AES_128_CBC_SHA, TLS_ECDHE_ECDSA_WI
      TH_AES_128_CBC_SHA, TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA,
      TLS_RSA_WITH_AES_128_CBC_SHA

    </SslCiphers>

    <MasterPolicy>node</MasterPolicy>

    <Group>group1</Group>

  </Common>

  <Log>

    <!-- nothing, error, warn, info, debug, dump. >

    < error is the default -->

    <Level>warn</Level>

  </Log>

  <Listen>

    <Port>26379</Port>

    <Threads>4</Threads>

    <!-- 0: telnet; 1: SSL; 2: password; 3: SSL + password. -->

    <Secure>0</Secure>

    <IsPlainPassword>true</IsPlainPassword>

    <!-- <Password>454d?51192b1704c60e19734ce6b38203</Password>-->

    <Password>123</Password>
```

```

</Listen>

<Center>

    <Password>454d51192b1704c60e19734ce6b38203</Password>

    <EndPoint>

        <Host>192.168.0.86</Host>

        <Port>6300</Port>

    </EndPoint>

</Center>

<!-- 哨兵集群配置（用于 sentinel sentinels 命令） -->

<Sentinels>

    <Sentinel>

        <Host>192.168.0.10</Host>

        <Port>26379</Port>

    </Sentinel>

</Sentinels>

<Services>

    <WebSession>

        <!-- 0: telnet; 1: SSL; 2: password; 3: SSL + password. -->

        <Secure>0</Secure>

        <IsPlainPassword>true</IsPlainPassword>

        <Password>123</Password>

        <EndPoints>192.168.0.10:6379, 192.168.0.20:6379</EndPoints>

    </WebSession>

</Services>

</Server>

```

参数说明：

通用配置：

- **Server.Common.SslCiphers:** 当启用 SSL 连接时，指定密钥交换使用的算法。早期版本的 SSL 存在“SSL/TLS 服务器瞬时 Diffie-Hellman 公共密钥过弱”漏洞。可通过此配置禁止使用 Diffie-Hellman 算法。建议配置：

```
TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384, TLS_ECDH_ECDSA_WITH_AES_256_GCM_SHA384, TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA, TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA, TLS_RSA_WITH_AES_256_CBC_SHA, TLS_KRB5_WITH_DES_CBC_MD5, TLS_KRB5_WITH_3DES_EDE_CBC_SHA, TLS_ECDHE_RSA_WITH_3DES_EDE_CBC_SHA, TLS_ECDH_ECDSA_WITH_AES_128_CBC_SHA, TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA, TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA, TLS_RSA_WITH_AES_128_CBC_SHA
```

- **Server.Common.MasterPolicy:** 被监控的 Rds 服务节点组中主节点的判断策略。缺省时（或配置为“node”）由被监控的服务节点的实际状态决定；当配置为“config”时将根据哨兵的配置文件中服务节点的配置顺序决定，配置顺序靠前的活着的节点为主节点。
- **Server.Common.Instance:** 当前进程的名称，配置后会在管理器的哨兵节点信息中查询到 instance 属性，用于控制台唯一标识节点。
- **Server.Common.Group:** 当前进程所属服务组的名字，配置后会在管理器的哨兵节点信息中查询到 group 属性，用于控制台归类节点。

日志配置:

- **Server.Log.Level:** 日志级别，缺省为“warn”。
- **Server.Log.BackDates:** 日志保存天数，超过时间的日志会被删除，缺省为 0（不删除），1 为只保留当天日志，以此类推。

Listen 配置:

哨兵模块监听端口、登录哨兵时的密码等配置（登录哨兵必须要密码认证，不允许无认证方式登录）。

- **Server.Listen.Port:** 哨兵监听的端口，缺省为 26379。
- **Server.Listen.Secure:** 是否开启 ssl 安全连接。当前示例中客户端采用普通连接。
- **Server.Listen.Password:** 登录哨兵的密码。当前示例中密码为 123。
- **Server.Listen.IsPlainPassword:** 保存的密码是否被加密过，true 为明文。

Center 配置:

中心节点配置用于哨兵节点连接中心节点上报运行状态信息等。

- **Server.Center.Password:** 连接中心节点的密码，SM4 加密。
- **Server.Center.EndPoint.Host:** 中心节点主机地址。
- **Server.Center.EndPoint.Port:** 中心节点服务端口号。

Sentinels 配置:

Sentinels 部分用于配置哨兵组中的其他哨兵的地址，目前用于“SENTINEL snetinals master”等命令时获取其他独立哨兵的信息（多台独立哨兵可以配置相同，程序自动判断并标记指向自己的配置项）。

程序会采用自身 Listen 节中配置的安全等级尝试连接其他哨兵程序，因此各哨兵的安全等级和密码等配置项必须相同。

- **Server.Sentinels.Sentinel.Host:** 其他哨兵程序的服务器 IP 地址。
- **Server.Sentinels.Sentinel.Port:** 其他哨兵程序使用的端口。

Services 配置:

Services 部分为被监控的服务的配置，其中的配置都是针对 RDS-Node 服务节点。Services 下可同时配置多个服务，每个服务配置一段，名称对应 RDS 的服务名(cfg.xml 中的 Server.Common.Service 配置项)。举例中配置了一个名为 WebSession 的服务。配置项含义如下：

- **Server.Services.WebSession.Secure:** 安全等级，和 RDS 服务节点 cfg.xmlServer.Listen.Secure 含义相同，此配置项需要与被连接的 RDS 的配置值相同。
- **Server.Services.WebSession.Username:** 连接 RDS 服务节点认证时采用的用户名。当 RDS 启用了 ACL 认证时出现该参数，启用 ACL 认证需要使用用户名、密码登录，本配置用于指定登录 RDS 的用户名。由于哨兵需要通过 PING 命令判断 RDS 节点是否正常，因此 ACL 需要配置该用户执行 PING 命令的权限。（**注意：如认证只需要密码则不能配置该项**）
- **Server.Services.WebSession.Password:** 连接 RDS 服务节点认证时采用的密码，如果 Secure 配置需要认证则该项不能为空。
- **Server.Services.WebSession.IsPlainPassword:** 上述配置的密码是否为明文，如果不是明文程序再需要使用密码时会首先尝试解密。
- **Server.Services.WebSession.EndPoints:** RDS 服务节点的网络地址，格式为 IP:Port，同时配多项用“.”分隔。

4.1.3 部署服务节点

1. 获取服务节点安装包，将安装包分别上传到 2 台服务器的/home/rds 目录下。
2. 进入/home/rds 目录，执行解压命令：

```
tar zxvf TongRDS-2.2.x.x.Node.tar.gz
```

3. 分别进入 2 台服务器的配置文件目录 pmemdb/etc

```
cd pmemdb/etc
```

4. 修改配置文件 dynamic.xml，2 台服务器上的配置都需要修改。

```
vi dynamic.xml
```

```
<Server>
  <Center>
    <Password>454d51192b1704c60e19734ce6b38203</Password>
    <EndPoint>
```

```
<Host>192.168.0.86</Host>

<Port>6300</Port>

</EndPoint>

</Center>

</Server>
```

参数说明：

- **Server.Center.Password:** 连接 Center 时的认证密码，示例中使用默认值即可。
- **Sever.Center.EndPoint:** 中心节点服务器 IP 地址，示例中修改 IP 地址为 192.168.0.86，端口为 6300。如果存在多台 Center 节点，EndPoint 可配置多条，例如：

```
<Server>

  <Center>

    <Password>454d51192b1704c60e19734ce6b38203</Password>

    <EndPoint>

      <Host>192.168.0.86</Host>

      <Port>6300</Port>

    </EndPoint>

    <EndPoint>

      <Host>192.168.0.87</Host>

      <Port>6300</Port>

    </EndPoint>

  </Center>

</Server>
```

4.1.4 部署中心节点

1. 获取中心节点安装包，将安装包上传到服务器的 **/home/rds** 目录下。
2. 进入 **/home/rds** 目录，执行解压命令：


```
tar zxvf TongRDS-2.2.x.x.MC.tar.gz
```
3. 将 License 文件 **center.lic** 复制到中心节点的 **pcenter** 目录下。
4. 进入配置文件目录 **pcenter/etc**

cd pcenter/etc

5. 在配置文件 **config.properties** 中检查如下配置项的值：

vi config.properties

```
...
# Service 节点连接 Center 节点使用的密码
server.password=454d51192b1704c60e19734ce6b38203
...
# 服务器监听端口，用于接收 MemDB 的上报数据
server.service.port=6300
...
server.sentinel.port=26379
...
```

- **server.password:** 节点接入时的认证密码，采用 SM4 加密，与服务节点 dynamic.xml 中的 Server.Center.Password 保持一致，示例中使用默认值即可。
- **server.service.port:** 中心节点的主服务端口，与服务节点 dynamic.xml 中的 Sever.Center.Endpoint.Port 保持一致，默认为 6300，示例中使用默认值即可。
- **server.sentinel.port:** Redis 哨兵的仿真接口，26379 为哨兵的缺省端口，示例中使用默认值即可。

6. 修改配置文件 **cluster.properties** 为如下配置：

vi cluster.properties

```
WebSession.type=sentinel
WebSession.nodes=2
WebSession.node0=192.168.0.10:6200
WebSession.node1=192.168.0.20:6200
```

- **WebSession.type=sentinel:** 服务“WebSession”（对应服务节点中的 Server.Service=WebSession 配置）的状态为哨兵模式。
- **WebSession.nodes:** 2 个服务节点的 IP 地址和端口（和实际的服务节点的运行情况对应）。

7. （可选）如果有多台中心节点集群，可在配置文件 **sync.properties** 中配置多台中心节点。例如配置 2 台 Center 集群：


```
sync.servers=2

sync.server1.host=192.168.0.86

sync.server1.port=6300

sync.server2.host=192.168.0.87

sync.server2.port=6300
```

当前示例中可不配置此文件。

4.1.5 启动服务

1. 在中心节点所在服务器的 **pcenter/bin** 目录下运行以下命令启动中心节点：

./StartCenter.sh

```
[rds@master bin]# ./StartCenter.sh

CenterService start at 6300

SentinelService start at 26379

Center start.

RestServer start at 8086
```

2. 分别在 2 台服务节点所在服务器的 **pmemdb/bin** 目录下运行以下命令启动服务节点：

./StartServer.sh

```
[rds@master bin]# ./StartServer.sh

load config-file '/home/rds/pmemdb/etc/cfg.xml' ok.


Server belonging to 'WebSession' is starting...

Memory cache create ok.

Start listening to port 6200

Start listening to port 6379


Server started.
```

服务节点启动成功后，各服务节点的 **dynamic.xml** 文件将被修改为类似如下配置：

```
<Server>

  <Center>

    <Password>454d51192b1704c60e19734ce6b38203</Password>

    <EndPoint>

      <Host>node1</Host>

      <Port>6300</Port>

    </EndPoint>

  </Center>

  <Synchronize>

    <EndPoint>

      <Host>192.168.0.10</Host>

      <Port>6200</Port>

    </EndPoint>

    <EndPoint>

      <Host>192.168.0.20</Host>

      <Port>6200</Port>

    </EndPoint>

  </Synchronize>

</Server>
```

3. 在哨兵节点所在服务器的 **sentinel/bin** 目录下运行以下命令启动哨兵节点：

./StartSentinel.sh

```
[rds@master bin]# ./StartSentinel.sh

SentiConfig::() load config-file '/home/rds/sentinel/etc/sentinel.xml' ok.

Begin to listen 26379

Sentinel start.
```

4.1.6 客户端jedis（以Version3.5.0为例）连接

按照上述配置，jedis 客户端采用如下参数连接（其中哨兵模块运行在 192.168.0.10:26389 上）：

```
JedisSentinelPool pool = new JedisSentinelPool("WebSession", new HashSet<String>() {{
    this.add("192.168.0.10:26379");
}}, (String)null, "123");

Jedis jedis = pool.getResource();

jedis.set("aaa", "ddd");

System.out.println("aaa = " + jedis.get("aaa"));

pool.close();

jedis.close();
```

其中构造 JedisSentinelPool 的最后 2 个参数分别是连接 RDS 服务节点的密码和连接哨兵的密码，用例中的参数含义为 RDS 不需要密码，哨兵密码是“123”。独立的哨兵模块也可以配置为无密码接入。

4.2 Center哨兵配置

4.2.1 部署前准备

Center 哨兵配置由中心节点管理 2 个服务节点，这 2 个服务节点工作在主备模式，并由 Center 中心节点提供哨兵功能。

在安装 TongRDS 前，需要在服务器上创建操作系统账号 rds，HOME 目录为/home/rds。

以下步骤中 TongRDS 均安装在此账号下。

TongRDSCenter 哨兵配置示例中使用的 IP 如下表所示：

部署类型	目录	IP	Port
中心节点	/home/rds/pcenter	192.168.0.86	6300
服务节点 1	/home/rds/pmemdb		TongRDS 协议端口：6200 Redis 仿真端口：6379
服务节点 2		192.168.0.87	哨兵监听端口：26379

4.2.2 部署服务节点

1. 获取服务节点安装包，将安装包分别上传到 2 台服务器的/home/rds 目录下。

2. 进入/home/rds 目录，执行解压命令：

```
tar zxvf TongRDS-2.2.x.x.Node.tar.gz
```

3. 分别进入 2 台服务器的配置文件目录 pmemdb/etc

cd pmemdb/etc

4. 修改配置文件 **dynamic.xml**，2 台服务器上的配置都需要修改。

vi dynamic.xml

```
<Server>

  <Center>

    <Password>454d51192b1704c60e19734ce6b38203</Password>

  <EndPoint>

    <Host>192.168.0.86</Host>

    <Port>6300</Port>

  </EndPoint>

</Center>

</Server>
```

- **Server.Center.Password:** 连接 Center 时的认证密码，示例中使用默认值即可。
- **Sever.Center.EndPoint:** 中心节点服务器 IP 地址，示例中修改 IP 地址为 192.168.0.86，端口为 6300。如果存在多台 Center 节点，EndPoint 可配置多条，例如：

```
<Server>

  <Center>

    <Password>454d51192b1704c60e19734ce6b38203</Password>

    <EndPoint>

      <Host>192.168.0.86</Host>

      <Port>6300</Port>

    </EndPoint>

    <EndPoint>

      <Host>192.168.0.87</Host>

      <Port>6300</Port>

    </EndPoint>

  </Center>

</Server>
```

4.2.3 部署中心节点

1. 获取中心节点安装包，将安装包上传到服务器的/home/rds 目录下。
2. 进入/home/rds 目录，执行解压命令：

```
tar zxvf TongRDS-2.2.x.x.MC.tar.gz
```

3. 将 License 文件 **center.lic** 复制到中心节点的 **pcenter** 目录下。
4. 进入配置文件目录 pcenter/etc

```
cd pcenter/etc
```

5. 在配置文件 **config.properties** 中检查如下配置项的值：

```
vi config.properties
```

```
...
# Service 节点连接 Center 节点使用的密码
server.password=454d51192b1704c60e19734ce6b38203
...
# 服务器监听端口，用于接收 MemDB 的上报数据
server.service.port=6300
...
server.sentinel.port=26379
...
```

- **server.password:** 节点接入时的认证密码，采用 SM4 加密，与服务节点 dynamic.xml 中的 Server.Center.Password 保持一致，示例中使用默认值即可。
- **server.service.port:** 中心节点的主服务端口，与服务节点 dynamic.xml 中的 Sever.Center.Endpoint.Port 保持一致，默认为 6300，示例中使用默认值即可。
- **server.sentinel.port:** Redis 哨兵的仿真接口，26379 为哨兵的缺省端口，示例中使用默认值即可。

6. 修改配置文件 **cluster.properties** 为如下配置：

```
vi cluster.properties
```

```
WebSession.type=sentinel
WebSession.nodes=2
WebSession.node0=192.168.0.86:6200
```

```
WebSession.node1=192.168.0.87:6200
```

- WebSession.type=sentinel: 服务 “WebSession”（对应服务节点中的 Server.Service=WebSession 配置）的状态为哨兵模式。
 - WebSession.nodes: 2 个服务节点的 IP 地址和端口（和实际的服务节点的运行情况对应）。
7. （可选）如果有多台中心节点集群，可在配置文件 **sync.properties** 中配置多台中心节点。例如配置 2 台 Center 集群：

```
sync.servers=2
sync.server0.host=192.168.0.86
sync.server0.port=6300
sync.server1.host=192.168.0.87
sync.server1.port=6300
```

当前示例中可不配置此文件。

4.2.4 启动服务

1. 在中心节点所在服务器的 **pcenter/bin** 目录下运行以下命令启动中心节点：

./StartCenter.sh

```
[rds@master bin]# ./StartCenter.sh
CenterService start at 6300
SentinelService start at 26379
Center start.
RestServer start at 8086
```

2. 分别在 2 台服务节点所在服务器的 **pmemdb/bin** 目录下运行以下命令启动服务节点：

./StartServer.sh

```
[rds@master bin]# ./StartServer.sh
load config-file '/home/rds/pmemdb/etc/cfg.xml' ok.

Server belonging to 'WebSession' is starting...

Memory cache create ok.
```

Start listening to port 6200

Start listening to port 6379

Server started.

服务节点启动成功后，各服务节点的 **dynamic.xml** 文件将被修改为类似如下配置：

```
<Server>
  <Center>
    <Password>454d51192b1704c60e19734ce6b38203</Password>
    <EndPoint>
      <Host>node1</Host>
      <Port>6300</Port>
    </EndPoint>
  </Center>
  <Synchronize>
    <EndPoint>
      <Host>192.168.0.86</Host>
      <Port>6200</Port>
    </EndPoint>
    <EndPoint>
      <Host>192.168.0.87</Host>
      <Port>6200</Port>
    </EndPoint>
  </Synchronize>
</Server>
```

4.2.5 部署后验证

4.2.5.1 测试目的

使用 jedis 客户端的哨兵模式接入，验证 RDS 中心节点模拟哨兵的功能；采用 jedis 多次接入，验证 RDS 模拟主节点功能。

本例使用 jedis 3.7.0（group: 'redis.clients', name: 'jedis', version: '3.7.0'）测试通过。

注：jedis 不同版本存在连接哨兵的 bug，例如 3.6.x 版本，无法采用有密码方式连接哨兵。如果测试不成功请首先检查 jedis 版本。

4.2.5.2 Jedis 接入（jedis 版本 3.7.0）

创建一个 java 类，输入如下代码：

```
public static void main(String args[]) {

    JedisSentinelPool pool = new JedisSentinelPool("WebSession",
new HashSet<String>() {{

        this.add("192.168.0.86:26379");

    }}, (String) null, "acioweor_483kja03np4h8238G");

    Jedis jedis = pool.getResource();

    jedis.set("aaa", "ddd");

    System.out.println("aaa = " + jedis.get("aaa"));

    jedis.close();

    pool.close();

}
```

其中：“WebSession”是服务名，需要与中心节点、服务节点的配置一致；“192.168.0.86:26379”为 Center 节点仿真哨兵的端口位置；“acioweor_483kja03np4h8238G”是哨兵接入的密码，对应 Center 的 active.properties 中的配置。

运行程序，在服务器 1（192.168.0.86）上观察到如下日志：

```
CacheServer::set() Set aaa<> = ddd ok
CacheServer::process_get() Get aaa = 'ddd' ok.
```

在服务器 2（192.168.0.87）上观察到如下日志：

```
CacheServer::sync() Sync aaa<> = ddd at 1629777554660 ok(0).
```


日志分析可知，jedis 从哨兵端口获得了主节点的访问端口，并成功完成读写操作，节点 2 获得同步数据。

注：Center 的哨兵功能不允许无密码接入，较低版本的 jedis 客户端无法使用。

4.2.5.3 验证主节点保持

继续上例测试，多次运行程序，观察读写操作均出现在节点 1 的日志中，节点 2 中始终是同步日志，说明正常情况下每次接入的操作均发生在一个节点上，另外的节点只负责备份。

4.2.5.4 备份节点异常测试

将备份节点杀掉。再次运行上述程序，观察主节点日志有正常的读写记录，说明服务正常。

将备份节点恢复，再次运行程序，读写仍然发生在主节点，说明备份节点的启动停止不会引起主备切换。

4.2.5.5 主节点异常测试

将主节点杀掉，再次运行程序，程序可正常完成。观察 2 节点日志发现，set 和 get 的操作日志出现在备份节点，说明中心节点做了主备切换。

将主节点恢复（等待其启动完成），再次运行程序，程序可正常完成。观察节点日志，set 和 get 操作的日志出现在主节点（节点 1），节点 2 上仍然是同步日志，说明 Center 将主节点切换回了节点 1。

第5章 故障排除

5.1 主备（主从）节点数据不同步

当配置完系统测试时发现主备（主从）节点不能同步数据，可从以下几个方面检查：

1. 检查各节点是否有 license 授权（使用 info 命令查看 Server 节的 license 内容），无授权的节点不能进行主备（主从）数据同步；
2. 检查各节点 cfg.xml 中的 Listen 中的 Secure 配置和 Password 配置是否一致；
3. 检查 cfg.xml 中 Sync 的配置，ListNumbers 或 ListLength 是否配成了 0；
4. 查 dynamic.xml 中 Synchronize 配置列表中是否有需要同步的全部节点（如果连接的 Center，不能直接改此处配置，需要改 Center 配置）；
5. 检查主、备（主、从）节点上的 cfg.xml 中的 BinaryCompatible 和 BinaryCompatibleKey 配置是否一致。

附录A 术语说明

名称	说明
TongRDS (简称 RDS)	东方通分布式内存数据缓存中间件
中心节点	提供服务节点注册、运行模式管理功能的管理节点
服务节点	实际提供数据缓存服务的功能节点
哨兵节点	用于监控服务节点主、备运行及健康监控的哨兵服务
代理节点	提供数据的动态负载均衡和接入协议的隔离的节点
集群模式	一种具备主从节点备份功能的数据分片运行模式
哨兵模式	由哨兵节点监控的全量数据主备服务节点备份功能的运行模式
代理集群模式	一种具备代理主从节点备份功能的数据分片运行模式
主节点	提供读、写功能的服务节点
备（从）节点	备份数据并提供只读功能的服务节点

